

SKILL: Regulation Graph Visualizer & Rule Editor

Overview

Project: Визуализатор и редактор регламентов поверх Regulation Management API
Stack: TypeScript / React · React Flow · Cytoscape.js · Python backend · RAGU GraphRAG
Target: Веб-приложение для аналитиков и архитекторов семантических систем

Этот документ описывает архитектуру, модели данных, экраны, API-контракты и критерии готовности для разработчика (Claude Code / AI-агента). Следуй инструкциям ниже точно и последовательно.

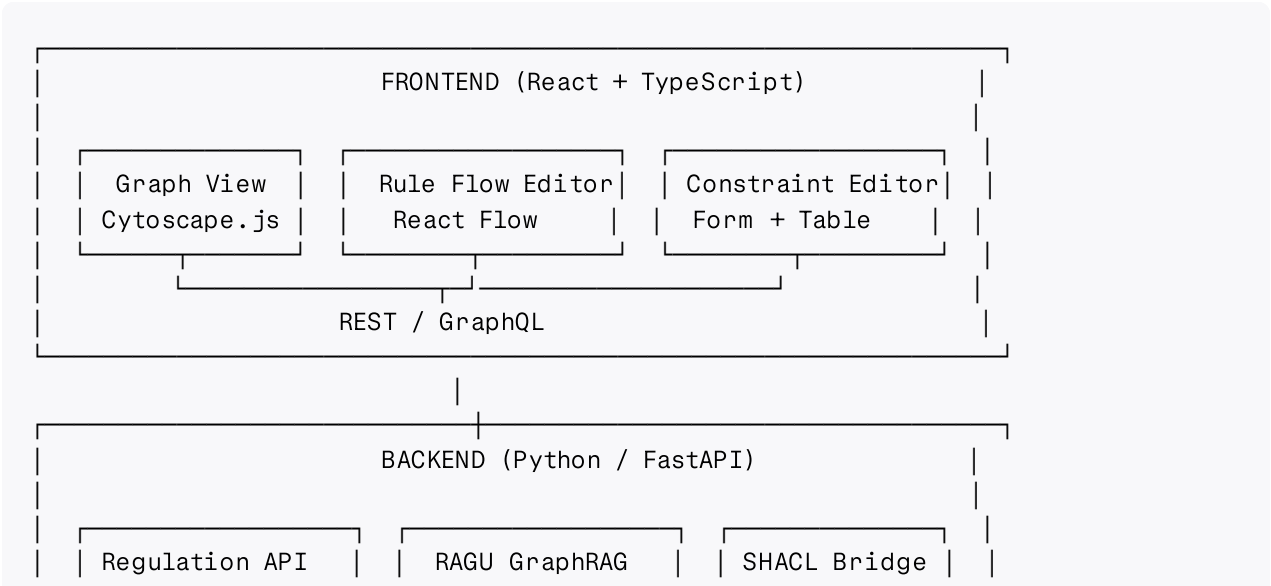
Context

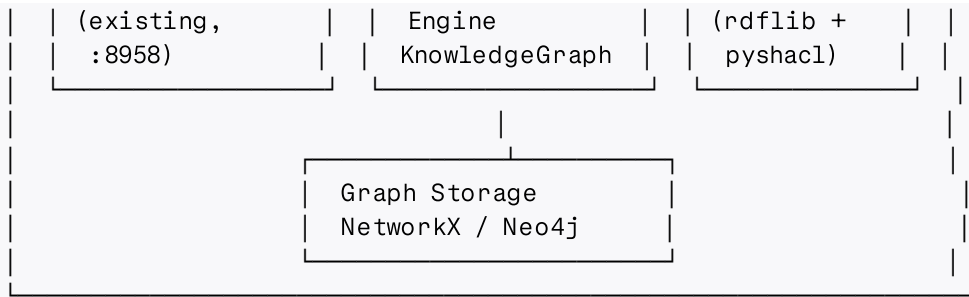
Существует REST API управления регламентами (<http://109.202.1.153:8958/docs>). Регламенты описывают набор правил контроля промышленных параметров (давление, диаметр, отклонения). Внутренняя семантическая модель основана на RDF/OWL + SHACL Shapes Constraint Language. Есть GraphRAG-движок RAGU для извлечения Entity/Relation из текстов регламентов.

Цель системы: дать аналитику возможность:

- 1. Видеть регламенты в виде графа знаний (knowledge graph view)
- 2. Редактировать правила в визуальном редакторе (node-based flow editor)
- 3. Работать с ограничениями SHACL через удобные формы (constraint editor)
- 4. Экспортировать/импортировать SHACL/RDF

Architecture





Domain Model

Core Entities

```
// Regulation – корневой объект регламента
interface Regulation {
  id: string;
  name: string;
  date: string;          // ISO 8601
  version: string;
  status: "active" | "draft" | "archived";
  parameters: Parameter[];
  constraints: Constraint[];
  recommendations: Recommendation[];
}

// Parameter – измеримый параметр
interface Parameter {
  id: string;
  name: string;          // "pressure" | "diameter" | ...
  datatype: "decimal" | "string" | "date" | "boolean";
  referenceValue: number;
  minInclusive?: number;
  maxInclusive?: number;
  deviationAllowed?: number;
  unit?: string;
}

// Constraint – SHACL-совместимое ограничение
interface Constraint {
  id: string;
  targetClass: string;   // "Regulation" | custom class
  path: string;          // sh:path
  datatype?: string;
  minCount?: number;
  maxCount?: number;
  minInclusive?: number;
  maxInclusive?: number;
  pattern?: string;
  message?: string;
  severity: "violation" | "warning" | "info";
}
```

```
// Recommendation – выходное действие при нарушении
interface Recommendation {
  id: string;
  condition: ConditionExpression;
  text: string;
  priority: 1 | 2 | 3;
  linkedParameters: string[]; // Parameter IDs
}
```

Rule DSL (JSON)

Каноническое хранение логики правил — JSON DSL:

```
{
  "rule_id": "rule_pressure_check",
  "regulation_id": "PressureAndDiameterRegulation",
  "nodes": [
    { "id": "n1", "type": "input",      "label": "pressure",  "paramRef": "pressure",
    { "id": "n2", "type": "threshold", "label": "ref ± dev",  "refValue": 20.5, "dev": 0.5,
    { "id": "n3", "type": "compare",   "label": "> threshold?", "operator": "outside",
    { "id": "n4", "type": "output",    "label": "Рекомендация", "action": "recommendation",
  ],
  "edges": [
    { "source": "n1", "target": "n2" },
    { "source": "n2", "target": "n3" },
    { "source": "n3", "target": "n4", "condition": "true" }
  ]
}
```

Node Types (Rule Flow Editor)

Каждый тип узла — отдельный React-компонент с handles и формой редактирования:

Node Type	Visual	Input handles	Output handles	Config fields
input	Зелёный, левый	—	1(data)	paramRef, label
threshold	Синий, ромб	1(value)	1(range)	refValue, deviation, unit
compare	Жёлтый, шестиугольник	1(value), 1 (range)	2 (true/false)	operator
formula	Фиолетовый	N(values)	1(result)	expression (JS-like)
switch	Оранжевый	1	N (cases)	cases[{label, value}]
output	Красный, правый	1	—	action, text, priority
shacl_constraint	Серый, пунктир	1	1	constraintRef

Screens

1. Graph View (/graph)

Описание: Обзорная карта всех регламентов и их связей.

Технология: Cytoscape.js + Cola layout

Элементы:

- Узлы: Regulation (синий), Parameter (зелёный), Constraint (серый), Recommendation (оранжевый), Source (светло-серый)
- Рёбра: has_parameter, applies_constraint, triggers, references
- Боковая панель: клик на узел → детали + кнопка "Открыть в редакторе"
- Toolbar: фильтр по типу узла, поиск по имени, кнопка экспорта PNG

Поведение:

- GET /api/graph → { nodes: CyNode[], edges: CyEdge[] }
- Узлы кластеризуются по regulation_id
- Double-click на Regulation → открыть Rule Flow Editor этого регламента

2. Rule Flow Editor (/regulations/:id/flow)

Описание: Визуальный редактор правил в виде блок-схемы.

Технология: React Flow + custom node components

Элементы:

- Canvas с drag-and-drop узлов
- Панель типов узлов (слева) — drag onto canvas
- Правая панель свойств — конфигурация выбранного узла
- Toolbar: Save, Validate, Export JSON, Export SHACL, Version History

Поведение:

- Загрузка: GET /api/regulations/:id/flow → Rule DSL JSON
- Сохранение: PUT /api/regulations/:id/flow → Rule DSL JSON
- Валидация: POST /api/regulations/:id/validate → { valid: bool, errors: ValidationError[] }
- При ошибках — подсветка проблемных узлов красным
- Автосохранение каждые 30 секунд в localStorage draft

3. Constraint Editor (/regulations/:id/constraints)

Описание: Табличный редактор SHACL-ограничений.

Элементы:

- Таблица: path | datatype | minCount | minInclusive | maxInclusive | pattern | severity | message
- Инлайн-редактирование ячеек
- Кнопка "Add Constraint" → форма снизу
- Кнопка "Import SHACL (Turtle)" → парсинг и merge в таблицу
- Кнопка "Export SHACL (Turtle)"

Поведение:

- GET /api/regulations/:id/constraints → Constraint[]
- PUT /api/regulations/:id/constraints → обновление всего набора
- POST /api/regulations/:id/shacl/import → multipart/turtle → merge
- GET /api/regulations/:id/shacl/export → text/turtle

4. Regulation List (/regulations)

Описание: Стартовый экран — список всех регламентов.

Элементы:

- Таблица: name | date | version | status | параметры (кол-во) | действия
- Действия: Graph View, Flow Editor, Constraints, Duplicate, Archive
- Кнопка "New Regulation" → wizard (3 шага: Meta → Parameters → Initial Constraints)
- Фильтр по статусу, поиск по имени

Backend API (новый слой поверх существующего)

Все эндпоинты — FastAPI, OpenAPI-документация автогенерируется.

Graph API

```
GET /api/graph
  → { nodes: CyNode[], edges: CyEdge[], meta: { total_nodes, total_edges } }

GET /api/graph/regulation/:id
  → subgraph для одного регламента
```

Flow API

```
GET /api/regulations/:id/flow
→ RuleDSL

PUT /api/regulations/:id/flow
body: RuleDSL
→ { ok: true, version: string }

POST /api/regulations/:id/validate
body: RuleDSL
→ { valid: bool, errors: ValidationError[] }

GET /api/regulations/:id/flow/history
→ FlowVersion[]
```

SHACL API

```
GET /api/regulations/:id/shacl/export
→ text/turtle

POST /api/regulations/:id/shacl/import
body: multipart/form-data (file: .ttl)
→ { merged_constraints: number, conflicts: ConflictReport[] }
```

Search API (RAGU)

```
POST /api/search
body: { query: string, mode: "local" | "global" | "naive" }
→ { response: string, entities: Entity[], sources: Chunk[] }
```

RAGU Integration

RAGU используется как **фоновый сервис** для:

1. **Автоматического построения графа** при загрузке/изменении регламентов
2. **Семантического поиска** по содержимому правил
3. **Извлечения связей** между регламентами из текстовых описаний

Инициализация

```
# ragu_service.py
from ragu import KnowledgeGraph, BuilderArguments, Settings
from ragu.models.llm import LLMOpenAI
from ragu.models.embedder import EmbedderOpenAI

Settings.language = "russian"
```

```

Settings.storage_folder = "./ragu_regulations_graph"

builder_settings = BuilderArguments(
    use_llm_summarization=True,
    make_community_summary=True,
    remove_isolated_nodes=True,
)

```

Доменные типы

```

# domain/graph_types.py
from ragu.graph.graph_index import Entity, Relation

class RegulationEntity(Entity):
    regulation_id: str
    parameter_name: str | None = None
    threshold: float | None = None
    entity_class: str = "Regulation"

class ConstraintRelation(Relation):
    condition_type: str # "range" | "equal" | "shacl_property"
    severity: str = "violation"

```

Конвертер графа для Cytoscape.js

```

# adapters/cytoscape_adapter.py

def to_cytoscape(knowledge_graph) -> dict:
    nodes = []
    edges = []

    for entity in knowledge_graph.get_all_entities():
        nodes.append({
            "data": {
                "id": entity.id,
                "label": entity.name,
                "type": entity.entity_type,
                "description": entity.description,
            }
        })

    for relation in knowledge_graph.get_all_relations():
        edges.append({
            "data": {
                "id": f"{relation.source_id}_{relation.target_id}",
                "source": relation.source_id,
                "target": relation.target_id,
                "label": relation.relation_type,
                "weight": relation.confidence,
            }
        })

```

```
return { "nodes": nodes, "edges": edges }
```

Frontend Stack

```
react@18
react-flow@11      # node-based flow editor
cytoscape@3        # graph visualization
cytoscape-cola     # force-directed layout
@tanstack/react-query # server state
zustand            # client state
tailwindcss@4      # styling
lucide-react       # icons
```

Project structure

```
src/
├─ app/
│  └─ routes/
│     └─ regulations/
│        └─ index.tsx      # List
│        └─ [id]/
│           └─ flow.tsx    # Rule Flow Editor
│           └─ constraints.tsx # Constraint Editor
│       └─ graph.tsx      # Graph View
├─ components/
│  └─ flow/
│     └─ nodes/
│        └─ InputNode.tsx
│        └─ ThresholdNode.tsx
│        └─ CompareNode.tsx
│        └─ FormulaNode.tsx
│        └─ SwitchNode.tsx
│        └─ OutputNode.tsx
│     └─ FlowCanvas.tsx
│     └─ NodePalette.tsx
│  └─ graph/
│     └─ GraphCanvas.tsx
│     └─ NodeDetailPanel.tsx
│  └─ constraints/
│     └─ ConstraintTable.tsx
│     └─ ShaclImportModal.tsx
├─ lib/
│  └─ api.ts      # typed API client
│  └─ rulesDsl.ts # DSL serialize/deserialize
│  └─ shaclBridge.ts # DSL → SHACL turtle
└─ store/
   └─ flowStore.ts
   └─ graphStore.ts
```


Data Flow Diagrams

Загрузка и редактирование правила

```
User opens /regulations/:id/flow
↓
GET /api/regulations/:id/flow
↓
RuleDSL JSON → deserialize → React Flow nodes/edges
↓
User edits nodes in canvas
↓
onChange → zustand flowStore
↓
Autosave (30s) → localStorage draft
↓
User clicks "Save"
↓
serialize → RuleDSL JSON
↓
POST /api/regulations/:id/validate → ok / errors
↓ (if ok)
PUT /api/regulations/:id/flow
↓
RAGU background job: rebuild subgraph
```

SHACL экспорт

```
User clicks "Export SHACL"
↓
GET /api/regulations/:id/shacl/export
↓
Backend: RuleDSL → rdflib Graph → Turtle serialization
↓
Download .ttl file
```

SHACL Bridge

Мэппинг между Rule DSL и SHACL выполняется Python-сервисом:

```
# shacl_bridge.py
from rdflib import Graph, Namespace, Literal
from rdflib.namespace import SH, XSD, RDF

SH_NS = Namespace("http://www.w3.org/ns/shacl#")
REG_NS = Namespace("http://regulations.local/ontology#")

def constraint_to_shacl(constraint: Constraint) -> Graph:
    g = Graph()
    shape = REG_NS[f"Shape_{constraint.id}"]
```

```

g.add((shape, RDF.type, SH.NodeShape))
g.add((shape, SH.targetClass, REG_NS[constraint.targetClass]))

prop = REG_NS[f"Prop_{constraint.id}"]
g.add((shape, SH.property, prop))
g.add((prop, SH.path, REG_NS[constraint.path]))

if constraint.datatype:
    g.add((prop, SH.datatype, XSD[constraint.datatype]))
if constraint.minInclusive is not None:
    g.add((prop, SH.minInclusive, Literal(constraint.minInclusive, datatype=XSD.
if constraint.maxInclusive is not None:
    g.add((prop, SH.maxInclusive, Literal(constraint.maxInclusive, datatype=XSD.
if constraint.minCount is not None:
    g.add((prop, SH.minCount, Literal(constraint.minCount, datatype=XSD.integer
if constraint.message:
    g.add((prop, SH.message, Literal(constraint.message, lang="ru")))

return g

```

Validation Rules

При валидации Rule DSL (endpoint POST /validate) проверяется:

1. **Graph connectivity** — нет изолированных узлов (кроме `type=input` и `type=output`)
2. **Completeness** — ровно один `input` → один `output` путь
3. **Type safety** — `compare` принимает `decimal` на оба входа
4. **Reference integrity** — `paramRef` в `input`-узлах существуют в `regulation.parameters`
5. **Threshold bounds** — `refValue ± deviation` не выходит за `[minInclusive, maxInclusive]` параметра
6. **Cycle detection** — нет циклических рёбер (DAG required)
7. **SHACL consistency** — если `shacl_constraint` узел ссылается на `constraint`, он должен существовать

Каждая ошибка возвращается как:

```

interface ValidationError {
    nodeId?: string;
    edgeId?: string;
    code: string;           // "ISOLATED_NODE" | "MISSING_OUTPUT" | ...
    message: string;
    severity: "error" | "warning";
}

```

Versioning

Каждое сохранение flow создаёт immutable snapshot:

```
interface FlowVersion {
  version_id: string;
  regulation_id: string;
  created_at: string; // ISO 8601
  author: string;
  comment?: string;
  dsl_snapshot: RuleDSL;
  diff_summary?: string; // "Added threshold node, updated deviation"
}
```

- GET /api/regulations/:id/flow/history → FlowVersion[]
- GET /api/regulations/:id/flow/history/:version_id → FlowVersion
- POST /api/regulations/:id/flow/restore/:version_id → restore snapshot

Design System

Применяется Nexus Design System (тёплые бежевые поверхности, teal accent).

Цветовая схема узлов в Rule Flow Editor:

Node Type	Fill	Border	Label color
input	--color-success-highlight	--color-success	--color-text
threshold	--color-blue-highlight	--color-blue	--color-text
compare	--color-gold-highlight	--color-gold	--color-text
formula	--color-purple-highlight	--color-purple	--color-text
switch	--color-orange-highlight	--color-orange	--color-text
output	--color-notification-highlight	--color-notification	--color-text
shacl_constraint	--color-surface-offset	--color-border	--color-text-muted

Cytoscape.js graph palette:

Node Class	Color
Regulation	--color-primary (teal)
Parameter	--color-success (green)

Node Class	Color
Constraint	<code>--color-text-muted</code> (gray)
Recommendation	<code>--color-orange</code>
Source	<code>--color-surface-dynamic</code>

Acceptance Criteria

Must Have (MVP)

- ☐ Graph View загружает и отображает все регламенты из API
- ☐ Клик на узел Regulation → открывается Rule Flow Editor
- ☐ Rule Flow Editor загружает DSL и рендерит правильные типы узлов
- ☐ Можно добавить/удалить/переконфигурировать узлы
- ☐ Сохранение через PUT `/api/regulations/:id/flow`
- ☐ Валидация с подсветкой ошибочных узлов
- ☐ Constraint Editor — просмотр и редактирование таблицы ограничений
- ☐ Export SHACL (Turtle) работает корректно
- ☐ Поддержка светлой/тёмной темы

Should Have

- ☐ Import SHACL с конфликт-репортом
- ☐ Version history — просмотр и восстановление
- ☐ Семантический поиск через RAGU (боковая панель)
- ☐ Автосохранение draft в localStorage
- ☐ Diff визуализация между версиями

Nice to Have

- ☐ Экспорт графа в PNG/SVG
- ☐ Approval workflow (draft → review → active)
- ☐ AI-подсказки при добавлении constraint (RAGU LocalSearch)
- ☐ Анимация при активации правил (simulation mode)

Implementation Order

1. Backend scaffold — FastAPI app, router structure, Pydantic models
2. RAGU integration — `ragu_service.py`, доменные типы, `cytoscape_adapter.py`
3. Graph API — `/api/graph` endpoint, Cytoscape JSON format
4. SHACL Bridge — `shacl_bridge.py`, import/export endpoints
5. Frontend scaffold — React + Vite + Tailwind + React Flow + Cytoscape.js
6. Graph View screen — Cytoscape canvas + NodeDetailPanel
7. Rule Flow Editor — custom nodes + canvas + property panel
8. Constraint Editor — table + inline editing + SHACL import modal
9. Validation layer — endpoint + frontend error highlighting
10. Versioning — snapshot storage + history UI

Environment Variables

```
# Backend
REGULATION_API_URL=http://109.202.1.153:8958
OPENAI_BASE_URL=...
OPENAI_API_KEY=...
RAGU_STORAGE_FOLDER=./ragu_regulations_graph
RAGU_LLM_MODEL=mistralai/mistral-medium-3
RAGU_EMBED_MODEL=emb-qwen/qwen3-embedding-8b

# Frontend
VITE_API_BASE_URL=http://localhost:8000
```

Key Constraints

- Все тексты интерфейса — **русский язык**
- RAGU language setting: `"russian"`
- Хранилище графа — NetworkX на старте, Neo4j при масштабировании
- SHACL валидация на бэкенде через `pyshacl`
- Rule DSL — единственный canonical формат; SHACL генерируется из DSL, не наоборот
- Версионирование: immutable snapshots в PostgreSQL/Supabase, не git
- Никакого localStorage для серверных данных — только draft-буфер flow